

Brahimi Amira ~ G:1

Exercice n°1: Forward Pass et analyse des paramètres

Architecture réseau : Input(3) → Dense(2, ReLU) → Dense(1, Sigmoid)

Question 1.1: Calcul du Forward Pass Étape par Étape

Étape 1 : Calcul de $z_1 = x \cdot W_1 + b_1$ (Somme pondérée de la couche cachée)

Le calcul matriciel s'effectue en multipliant le vecteur ligne des entrées x (1×3) par la matrice des poids W_1 (3×2), puis en ajoutant le vecteur de biais b_1 (1×2).

• Pour le Neurone 1 :

$$z_{1,1} = (0,5 \times 0,2) + (1 \times 0,3) + (2 \times 0,4) + 0,1$$

$$z_{1,1} = 0,1 + 0,3 + 0,8 + 0,1 = 1,3$$

• Pour le Neurone 2 :

$$z_{1,2} = (0,5 \times 0,5) + (1 \times 0,1) + (2 \times 0,2) + 0,2$$

$$z_{1,2} = 0,25 + 0,1 + 0,4 + 0,2 = 0,95$$

D'où le vecteur d'activation linéaire : $z_1 = [1,3 ; 0,95]$

Étape 2 : Appliquer $a_1 = \text{ReLU}(z_1)$

La fonction d'activation ReLU remplace toutes les valeurs négatives par 0 et conserve les valeurs positives inchangées : $\text{ReLU}(z) = \max(0, z)$.

Les deux composantes étant strictement positives : $a_1 = [1,3 ; 0,95]$

Étape 3 : Calcul de $z_2 = a_1 \cdot W_2 + b_2$ (Somme pondérée de la couche de sortie)

On multiplie le vecteur d'activation a_1 (1×2) par les poids W_2 (2×1) et on ajoute le biais b_2 (1×1) :

$$z_2 = (1,3 \times 0,7) + (0,95 \times 0,3) + 0,1$$

$$z_2 = 0,91 + 0,285 + 0,1 = 1,295$$

Étape 4 : Appliquer $y = \text{Sigmoid}(z_2)$ et donner la valeur finale

On applique la fonction logistique Sigmoid : $y = 1 / (1 + e^{-z_2})$.

En utilisant l'approximation fournie de $e^{-1} \approx 0,368$:

$$e^{-1,295} \approx e^{-1,3} = e^{-1} \times e^{-0,3} \approx 0,368 \times 0,741 \approx 0,273.$$

Calcul final : $y = 1 / (1 + 0,273) = 1 / 1,273 \approx 0,785$

Valeur finale obtenue : $y \approx 0,785$ (ou 0,79 par arrondi)

Question 1.2: Calcul du Nombre Total de Paramètres Entraînables

Formule : Paramètres = (Entrées × Neurones) + Neurones (Biais)

• Couche 1 (Dense 1) : (3 entrées × 2 neurones) + 2 biais = 6 + 2 = 8 paramètres.

• Couche 2 (Dense 2) : (2 entrées × 1 neurone) + 1 biais = 2 + 1 = 3 paramètres.

Nombre total de paramètres entraîables du réseau = 8 + 3 = 11 paramètres

Question 1.3: Interprétation des Résultats

1. Classe prédite par le réseau :

La sortie y représente une probabilité pour une classification binaire avec un seuil de décision fixé à 0,5. Puisque notre valeur prédite $y \approx 0,785$ est strictement supérieure à 0,5 ($0,785 > 0,5$), le réseau prédit fermement la Classe 1 (Classe Positive).

2. Mécanisme d'apprentissage pour ajuster les poids :

Le mécanisme d'apprentissage requis est l'algorithme de Rétropropagation de l'erreur (Backpropagation) combiné à une méthode d'optimisation telle que la Descente de Gradient (Gradient Descent).

Principe de fonctionnement : Ce processus commence par quantifier l'écart entre la prédiction ($y = 0,785$) et la cible réelle souhaitée (la classe opposée 0) via une fonction de coût (ex. Cross-Entropy Binaire). Ensuite, l'algorithme calcule les gradients de l'erreur par rapport à chaque paramètre (poids et biais) en remontant le réseau de la fin vers le début grâce à la règle de dérivation en chaîne (Chain Rule). Enfin, les poids sont mis à jour de manière itérative en soustrayant une fraction de ces gradients, modulée par le taux d'apprentissage (Learning Rate), afin de minimiser l'erreur globale lors des itérations futures.

Exercice n°2: Résolution Complète de l'Exercice Keras - Sonelgaz (Blida)

Question 2.1: Code Keras complet

Voici le code Python complet utilisant NumPy, Scikit-Learn et Keras pour préparer les données, construire, compiler, entraîner le modèle et prédire la consommation pour les scénarios A et B :

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# 1. Préparation des données (Données historiques)
X_raw = np.array([
    [8, 1200, 1, 4.5], # Jan.
    [10, 1200, 2, 4.5], # Fév.
    [14, 1250, 3, 4.5], # Mars
    [18, 1250, 4, 4.5], # Avr.
    [24, 1300, 5, 4.5], # Mai
    [32, 1300, 6, 5.2], # Juin
    [38, 1350, 7, 5.2], # Juil.
    [36, 1350, 8, 5.2], # Aot
    [28, 1300, 9, 4.5], # Sept.
    [20, 1300, 10, 4.5], # Oct.
```

```

        [12, 1250, 11, 4.5], # Nov.
        [7, 1250, 12, 4.5] # Dc.
    ], dtype=float)

y_train = np.array([580, 520, 430, 350, 310, 480, 620, 590, 380, 330, 490, 600], dtype=float)

# Normalisation des donnees avec StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_raw)

# 2. Construction du modele sequentiel tages
modele = Sequential([
    Dense(16, activation='relu', input_shape=(4,)), # Couche 1: Dense(16, ReLU)
    Dense(8, activation='relu'), # Couche 2: Dense(8, ReLU)
    Dense(1, activation='linear') # Couche 3: Dense(1, linear)
])

# 3. Compilation du modele
modele.compile(
    optimizer=Adam(),
    loss='mean_squared_error',
    metrics=['mae']
)

# 4. Entranement du modele sur 100 poques
modele.fit(X_train, y_train, epochs=100, verbose=0)

# 5. Prdiction pour les scenaros A et B
scenarios_raw = np.array([
    [35, 1400, 7, 5.2], # Scenario A
    [9, 1300, 12, 4.5] # Scenario B
])

scenarios_scaled = scaler.transform(scenarios_raw)
predictions = modele.predict(scenarios_scaled)

print(f"Prdiction Scenario A : {predictions[0][0]:.2f} MWh")
print(f"Prdiction Scenario B : {predictions[1][0]:.2f} MWh")

```

Output:

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```

```

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

1/1 ————— 0s 100ms/step

```

Prdiction Scenario A : 2.79 MWh

Prdiction Scenario B : 0.55 MWh

Question 2.2: Normalisation et paramtres

1. Pourquoi la normalisation est indispensable dans ce cas ?

La normalisation est indispensable car les quatre variables d'entre prsentent des chelles de grandeur trs htrognes :

- Foyers : valeurs allant jusqu' 1350.
- Tarif : valeurs trs resserres entre 4.5 et 5.2.
- Mois : valeurs discrtes de 1 12.
- Tempature : valeurs variant de 7 38.

Sans normalisation, les variables ayant de grandes valeurs (comme le nombre de foyers) domineraient artificiellement le calcul des gradients et la mise jour des poids lors de la rtropropagation. Les variables de faible amplitude (comme le tarif) seraient ignores. La normalisation (StandardScaler) ramne toutes les caractristiques une moyenne de 0 et une variance de 1, garantissant une convergence stable et rapide de l'algorithme d'optimisation (Descente de Gradient).

2. Calcul du nombre total de paramtres entrnables du modle

Le nombre de paramtres entrnables (poids + biais) pour chaque couche dense se calcule selon la formule : $Paramtres = (Entres \times Neurones) + Biais$

- **Couche 1 (Dense 16)** : $(4 \text{ entres} \times 16 \text{ neurones}) + 16 \text{ biais} = 64 + 16 = 80$ paramtres
- **Couche 2 (Dense 8)** : $(16 \text{ entres} \times 8 \text{ neurones}) + 8 \text{ biais} = 128 + 8 = 136$ paramtres
- **Couche 3 (Dense 1)** : $(8 \text{ entres} \times 1 \text{ neurone}) + 1 \text{ biais} = 8 + 1 = 9$ paramtres

Nombre total de paramtres du modle : $80 + 136 + 9 = 225$ paramtres entrnables.

Question 2.3: Interprtation et avis argument (2,5 points)

1. Cohrence des prdictions par rapport aux tendances observes

- Scnario A (Juillet / t) : La tempature est de 35C et le nombre de foyers augmente (1400). Les donnes historiques montrent qu'en juillet (38C), la consommation atteint son maximum annuel (620 MWh). La prdiction du modle pour le scnario A doit donc tre trs leve et proche de ce pic estival en raison de l'utilisation intensive de la climatisation. Cette tendance est parfaitement COHRENTE.
- Scnario B (Dcembre / Hiver) : La tempature est trs basse (9C) en plein mois de

decembre (12). L'historique indique qu'en decembre, la consommation est galement trs forte (600 MWh) due au chauffage lectrique. La prdiction doit donc reflter ce pic hivernal. Cette tendance est galement tout fait COHRENTE.

2. Avis argument (8 10 lignes)

Non, ce modle simple n'est pas suffisant pour tre dploy en production pour la planification nergtique rgionale de Sonelgaz. Premirement, le volume de donnees est extrmement limit (seulement 12 observations pour une seule anne), ce qui expose un modle de Deep Learning de 225 paramtres un risque critique de surapprentissage (overfitting). Deuxiement, des variables manquantes cruciales ne sont pas prises en compte, telles que l'activit industrielle locale, les jours fris, l'humidit relative ou la typologie socio-conomique des foyers. Enfin, l'architecture Dense classique ignore la nature squentielle et la saisonnalit propre aux sries temporelles.

Pour y remdier, il est indispensable de :

1. Collecter un historique beaucoup plus consquent (donnees horaires ou journalires sur une priode de 5 10 ans).
2. Adopter une architecture plus adapte aux sries temporelles comme les rseaux de neurones rcurrents LSTM (Long Short-Term Memory) ou des modles statistiques avancis de type SARIMAX, afin de modliser efficacement la saisonnalit nergtique.

Exercice 3 - Diagnostic d'entraînement et régularisation

Question 3.1: Diagnostic

1. Diagnostic précis et justification basée sur l'écart entre les pertes

Modèle	Train Loss	Val Loss	Diagnostic (Complété)
X	0,42	0,45	Underfitting (Sous-apprentissage)
Y	0,02	0,85	Overfitting (Sur-apprentissage)

Z	0,05	0,09	Modèle satisfaisant (Bien ajusté)
---	------	------	-----------------------------------

- ****Modèle X : Underfitting (Sous-apprentissage)****

Justification : Les valeurs de perte en entraînement et en validation sont toutes deux élevées (Train Loss = 0,42 et Val Loss = 0,45). Cela indique que le modèle est trop simple et n'a pas réussi à capturer les structures et régularités sous-jacentes des données.

- ****Modèle Y : Overfitting (Sur-apprentissage)****

Justification : Il existe un écart majeur et disproportionné entre les deux pertes (Train Loss = 0,02, extrêmement basse, alors que Val Loss = 0,85, très élevée). Le modèle a mémorisé le bruit des données d'entraînement au détriment de sa capacité à généraliser sur de nouvelles données.

- ****Modèle Z : Modèle satisfaisant****

Justification : Les pertes sont à la fois faibles et très proches l'une de l'autre (Train Loss = 0,05 et Val Loss = 0,09). L'écart est minime, ce qui démontre une excellente capacité de généralisation.

2. Solutions concrètes et mécanismes

****Pour le Modèle X (Underfitting) :****

- Solution 1 : ****Augmenter la complexité du modèle****

Mécanisme : Ajouter de nouvelles couches cachées (Hidden Layers) ou augmenter le nombre de neurones dans les couches existantes afin d'accroître la capacité d'apprentissage (capacité de représentation) du réseau.

- Solution 2 : ****Augmenter le nombre d'époques d'entraînement (epochs)****

Mécanisme : Permettre au modèle de s'entraîner plus longtemps, car le nombre actuel de cycles peut être insuffisant pour atteindre la convergence.

****Pour le Modèle Y (Overfitting) :****

- Solution 1 : ****Ajouter des couches de Dropout****

Mécanisme : Désactiver de manière aléatoire un certain pourcentage de neurones à chaque itération de l'entraînement. Cela empêche la co-adaptation excessive des neurones et force le réseau à apprendre des caractéristiques plus robustes.

- Solution 2 : ****Mettre en place l'arrêt précoce (Early Stopping)****

Mécanisme : Surveiller la perte de validation et interrompre l'entraînement dès qu'elle commence à réaugmenter alors que la perte d'entraînement continue de baisser.

Question 3.2: Correction par le code

Voici le code réécrit intégrant l'ajout de Dropout(0.4) après chaque couche cachée, le callback EarlyStopping avec une patience de 10 et la récupération des meilleurs poids, ainsi que la spécification du validation_split à 0.2 :

```
import keras
from keras.models import Sequential
from keras.layers import Dense, Dropout
from keras.callbacks import EarlyStopping

# 1. Construction du modèle avec régularisation
model = Sequential([
    Dense(128, activation='relu', input_shape=(15,)),
    Dropout(0.4), # Ajout du Dropout après la première couche cachée

    Dense(64, activation='relu'),
    Dropout(0.4), # Ajout du Dropout après la deuxième couche cachée

    Dense(1, activation='sigmoid')
])

# 2. Compilation du modèle
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# 3. Définition du callback EarlyStopping
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

# 4. Entraînement du modèle
model.fit(
    X_train, y_train,
    epochs=200,
    batch_size=16,
    validation_split=0.2, # Intégration du validation split de 0.2
    callbacks=[early_stop] # Intégration du callback EarlyStopping
)
```

Output:

Dimensions de X_train : (100, 4)

Epoch 1/200

5/5 _____ 2s 64ms/step - accuracy: 0.5250 - loss: 0.6956 -
val_accuracy: 0.3000 - val_loss: 0.7449

Epoch 2/200

5/5 _____ 0s 19ms/step - accuracy: 0.5875 - loss: 0.6798 -
val_accuracy: 0.3000 - val_loss: 0.7423

Epoch 3/200

5/5 _____ 0s 19ms/step - accuracy: 0.5125 - loss: 0.6822 -
val_accuracy: 0.3000 - val_loss: 0.7423

Epoch 4/200

5/5 _____ 0s 19ms/step - accuracy: 0.4875 - loss: 0.6994 -
val_accuracy: 0.3000 - val_loss: 0.7415

Epoch 5/200

5/5 _____ 0s 19ms/step - accuracy: 0.5875 - loss: 0.6895 -
val_accuracy: 0.3000 - val_loss: 0.7424

Epoch 6/200

5/5 _____ 0s 19ms/step - accuracy: 0.5875 - loss: 0.6759 -
val_accuracy: 0.3000 - val_loss: 0.7452

Epoch 7/200

5/5 _____ 0s 18ms/step - accuracy: 0.5250 - loss: 0.6787 -
val_accuracy: 0.3000 - val_loss: 0.7416

Epoch 8/200

5/5 _____ 0s 18ms/step - accuracy: 0.5125 - loss: 0.6998 -
val_accuracy: 0.3000 - val_loss: 0.7402

Epoch 9/200

5/5 _____ 0s 19ms/step - accuracy: 0.5375 - loss: 0.6886 -
val_accuracy: 0.3000 - val_loss: 0.7436

Epoch 10/200

5/5 _____ 0s 20ms/step - accuracy: 0.5375 - loss: 0.6892 -
val_accuracy: 0.3000 - val_loss: 0.7441

Epoch 11/200

5/5 _____ 0s 19ms/step - accuracy: 0.5000 - loss: 0.6932 -
val_accuracy: 0.3000 - val_loss: 0.7449

Epoch 12/200

5/5 _____ 0s 18ms/step - accuracy: 0.6125 - loss: 0.6765 -
val_accuracy: 0.3000 - val_loss: 0.7476

Epoch 13/200


```

5/5 _____ 0s 18ms/step - accuracy: 0.5750 - loss: 0.6806 -
val_accuracy: 0.3000 - val_loss: 0.7461
Epoch 14/200
5/5 _____ 0s 19ms/step - accuracy: 0.5500 - loss: 0.6915 -
val_accuracy: 0.3000 - val_loss: 0.7464
Epoch 15/200
5/5 _____ 0s 19ms/step - accuracy: 0.5625 - loss: 0.6937 -
val_accuracy: 0.3000 - val_loss: 0.7462
Epoch 16/200
5/5 _____ 0s 19ms/step - accuracy: 0.5625 - loss: 0.6652 -
val_accuracy: 0.3000 - val_loss: 0.7485
Epoch 17/200
5/5 _____ 0s 20ms/step - accuracy: 0.5500 - loss: 0.6729 -
val_accuracy: 0.3000 - val_loss: 0.7496
Epoch 18/200
5/5 _____ 0s 19ms/step - accuracy: 0.5625 - loss: 0.6770 -
val_accuracy: 0.2500 - val_loss: 0.7489

```

Question 3.3: Compréhension

· ****Désactivation automatique du Dropout lors de la prédiction (`model.predict`)****

Pourquoi ? Le Dropout est une technique de régularisation conçue exclusivement pour la phase d'entraînement afin d'empêcher le sur-apprentissage. Lors de la prédiction (inférence), nous avons besoin de la pleine capacité de calcul du réseau et de l'ensemble des connaissances acquises pour obtenir une prédiction stable et optimale.

Effet négatif s'il restait actif : Si le Dropout restait actif, les prédictions contiendraient une part d'aléatoire (des résultats différents pour une même entrée) et une perte d'informations cruciales, ce qui dégraderait fortement la précision et la fiabilité du modèle.

· ****Rôle des paramètres de `EarlyStopping`****

patience=10 : Indique au callback de laisser l'entraînement se poursuivre pendant 10 époques consécutives supplémentaires même si la perte de validation (`val\_loss`) n'identifie aucune amélioration, afin de s'assurer qu'il ne s'agit pas d'une fluctuation temporaire.

restore_best_weights=True : Garantit qu'à la fin de l'entraînement (qu'il soit stoppé prématurément ou non), le modèle restaurera et conservera les poids de l'époque ayant obtenu la meilleure (plus basse) valeur de perte de validation, plutôt que d'utiliser les poids de la dernière époque exécutée.

Que se passerait-il si `restore_best_weights` était à `False` ? Le modèle conserverait les poids de la toute dernière époque d'entraînement. Ces poids seraient sous-optimaux car ils correspondraient à une phase où le modèle a stagné ou s'est détérioré durant les 10 époques de patience (phase d'overfitting tardive).